

Documentacion Tecnica PUA

Especificacion de Arquitectura, Modelo de Datos, APIs y Pruebas

Plataforma Integral de Registro, Georreferenciacion y Procesos de Reeduccion

INFORMACION DEL DOCUMENTO

Version: 1.0.0 (Release Estable)
Fecha: August 2026
Elaborado por: Equipo de Desarrollo y Arquitectura PUA

RESUMEN Y CONFIDENCIALIDAD

Este documento contiene informacion operativa y tecnologica del Sistema PUA.
Su distribucion esta reservada para personal tecnico, administrativo y operativo.
Prohibida su reproduccion total o parcial sin autorizacion expresa.

1. Introduccion y Objetivos del Sistema

El Sistema PUA (Plataforma Unica de Agresores) es una solucion de software institucional disenada para la administracion integral, geolocalizacion, perfilamiento multidimensional y seguimiento operativo de procesos de reeducacion para personas agresoras. El sistema proporciona a las dependencias gubernamentales, coordinadores y facilitadores una plataforma segura, robusta y centralizada para la toma de decisiones basada en evidencia.

[INFO] Mision del Software

Estandarizar la recopilacion de datos sociodemograficos y psicologicos, optimizar la medicion de cumplimiento en procesos reeducativos y proporcionar georreferenciacion precisa mediante extensiones espaciales GIS.

1.1 Pila Tecnologica (Tech Stack)

Capa	Tecnologia	Version	Proposito Clave
Backend REST API	FastAPI (Python)	0.136+	Servidor asincrono, alto rendimiento y autod...
Capa ORM	SQLAlchemy	2.0.49	Mapeo objeto-relacional, mixins de auditoria...
Extension GIS	GeoAlchemy2 / PostGIS	0.20 / 4326	Soporte nativo para puntos geograficos EPSG:...
Serializacion	Pydantic v2	2.13+	Validacion estricta de esquemas, CURP y deco...
Frontend SPA	Angular (Standalone)	21.2.0	Arquitectura moderna orientada a componentes...
Base de Datos	PostgreSQL	16+ PostGIS	Persistencia relacional con integridad refer...

2. Arquitectura Global del Sistema

La aplicacion opera bajo una arquitectura desacoplada Cliente-Servidor (SPA + REST API) con separacion total de responsabilidades. El frontend se comunica exclusivamente mediante peticiones HTTPS autenticadas mediante tokens JWT.

- Capa de Presentacion (Angular 21): Estructura modular basada en componentes Standalone, carga perezosa de rutas (Lazy Loading) y servicios genericos reutilizables.
- Capa de Aplicacion (FastAPI): Routers modulares, inyeccion de dependencias para sesiones de base de datos y control de acceso basado en roles (RBAC).
- Capa de Datos (PostgreSQL/PostGIS): Diseno normalizado con soporte para borrado logico (Soft-Delete) y mixins transversales de auditoria.

3. Modelo de Datos y Esquema de Base de Datos

El esquema relacional implementa un patron de auditoria universal (AuditMixin) y catalogo maestro (CatalogoMixin). La entidad principal 'agresor' concentra la informacion general, mientras que las relaciones complejas se resuelven a traves de tablas pivote intermedias.

3.1 Entidades Principales

Tabla	Clave Primaria	Descripcion Operativa	Campos Relevantes
usuarios	id (SERIAL)	Usuarios del sistema	username, correo, hashed_password, rol...
roles	id (SERIAL)	Roles de acceso (RBAC)	nombre (SUPER_ADMIN, etc.), descripcio...
agresor	folio (SERIAL)	Expediente de agresores	curp, nombre, lugar_nacimiento/residen...
procesoreeduca...	folio (SERIAL)	Proceso judicial/reeducativo	agresor_id, fecha_inicio, denunciante,...
grupo	folio (SERIAL)	Sedes y grupos de apoyo	lugar, ubicacion (POINT SRID 4326), cr...
sesion	folio (SERIAL)	Catalogo de temas	nombre, objetivo pedagogico, created_at
lista	id (SERIAL)	Registro de asistencias	agresor_id, grupo_id, sesion_id, fecha

3.2 Mixin de Auditoria Transversal (AuditMixin)

Todas las tablas operativas heredan automaticamente los siguientes campos de trazabilidad para garantizar integridad:

- created_at / updated_at: Timestamps de creacion y modificacion con valor por defecto del servidor.
- created_by / updated_by: Claves foraneas referenciando a la tabla usuarios(id).
- deleted_at / deleted_by / is_deleted: Mecanismo de borrado logico que preserva la historia de datos.

4. Catalogo de APIs y Endpoints REST

Todos los endpoints operan bajo formato JSON (excepto /login que requiere x-www-form-urlencoded). La autentificacion se realiza mediante el encabezado 'Authorization: Bearer <token>'.

Metodo	Ruta Endpoint	Seguridad	Descripcion
POST	/login	Publico	Autentica credenciales y emite token JWT ...
GET	/agresores	Autenticado	Obtiene lista de agresores activos con da...
POST	/agresores	Autenticado	Crea un nuevo expediente validando CURP u...
GET	/agresores/{folio}	Autenticado	Consulta un expediente individual decodif...
PUT	/agresores/{folio}	Autenticado	Actualiza informacion sociodemografica y ...
DELETE	/agresores/{folio}	SUPER_ADMIN	Ejecuta borrado logico (is_deleted=True, ...
GET / POST	/proceso-reeducacion	Autenticado	Administra expedientes de reeducacion aso...
GET / PUT	/proceso-reeducacion/{folio}	Autenticado	Consulta y actualiza detalles de carpetas...
GET / POST	/grupos	Autenticado	Lista y registra sedes operativas con coo...
GET / POST	/sesiones	Autenticado	Gestiona el catalogo de temas y objetivos...
GET / POST	/lista	Autenticado	Registra y consulta asistencias de agreso...
CRUD	/catalogos/{tipo}	Autenticado/Admin	15 Catálogos dinamicos creados con patron...

4.1 Patron Factory en Routers de Catalogos

Para evitar la duplicacion de codigo en los 15 catalogos del sistema (adicciones, estados civiles, etc.), se implemento 'crear_router_catalogo()' en backend/app/routers/factory.py, que autogenera endpoints POST, GET (con filtro is_deleted), GET by ID, PUT y DELETE estandarizados con tipado Pydantic.

5. Arquitectura del Frontend (Angular 21)

El frontend utiliza Standalone Components, Reactive Forms y un patron de herencia generica para maximizar la mantenibilidad:

- BaseCrudService<TItem, T>: Servicio generico que provee metodos HTTP estandarizados (obtenerTodos, obtenerPorId, crear, actualizar, eliminar).
- BaseCrudComponent<TItem, T>: Directiva base abstracta que gestiona estado de carga, paginacion reactiva, busqueda en memoria, modales de confirmacion y control de formularios.
- Lazy Loading (app.routes.ts): Todas las vistas se cargan bajo demanda con loadComponent: () => import(...) para optimizar el bundle inicial.
- AuthInterceptor & AuthGuard: Inyeccion automatica del JWT en peticiones y proteccion de rutas con verificacion SSR-safe (isPlatformBrowser).

6. Cobertura de Pruebas Unitarias y Calidad

El proyecto cuenta con suites de pruebas automatizadas ejecutables de manera local y en pipelines CI/CD:

Capa	Herramienta	Archivos de Test	Pruebas	Estado
Backend Core	Python unittest	test_security.py	3 tests	100% OK (Hash bcrypt, JWT)
Backend Schemas	Python unittest	test_schemas.py	9 tests	100% OK (Validacion CURP, GIS)
Backend RBAC	Python unittest	test_role_verifier.py	2 tests	100% OK (Permisos 403/200)
Frontend Services	Vitest + HttpT...	8 service specs	25 tests	100% OK (BaseCrud y APIs)
Frontend Guards	Vitest	auth-guard/interceptor	4 tests	100% OK (JWT injection)
Frontend Views	Vitest + TestBed	12 component specs	11 tests	100% OK (Instanciacion y UI)

[EXITO] Comandos de Ejecucion de Pruebas

Backend: python3 -m unittest discover -s tests -p 'test_*.py' -v Frontend: npx vitest run